

MODUL DATA SCIENCE FOUNDATIONS

University of London — Spesialisasi 8 Course

Dari nol sampai bisa: konsep, tools, Python, statistik, dan satu proyek capstone utuh.

Disusun untuk: BAGHIZ ASSHIDDIQIE ARASYA

7 Oktober 2026

Peta 8 Course dalam Modul Ini

Modul ini merangkum seluruh spesialisasi **Data Science Foundations** dari University of London yang terdiri dari 8 course. Setiap bab memakai bahasa sehari-hari dan setiap istilah asing langsung dijelaskan dengan analogi saat pertama kali muncul, jadi bisa dipahami bahkan tanpa latar belakang teknis.

- **Bab 1 — The Data Science Profession:** apa itu data science, supervised vs unsupervised learning, klasifikasi, regresi, clustering.
- **Bab 2 — What is Data Science?:** big data 5V, Hadoop, data mining, machine learning vs deep learning, aplikasi dan karir.
- **Bab 3 — Tools for Data Science:** bahasa (Python, R, Julia, SQL), library (pandas, NumPy, Matplotlib, scikit-learn), Jupyter, Git/GitHub.
- **Bab 4 — Problems, Algorithms and Flowcharts:** problem, algoritma, simbol flowchart, pseudocode, algoritma Euclid dan metode Heron.
- **Bab 5 — Python for Data Science, AI & Development:** dasar Python: variabel, list, indexing, perulangan.
- **Bab 6 — Statistics and Clustering in Python:** mean/median/modus, varians, standar deviasi, populasi vs sampel, K-means.
- **Bab 7 — Capstone: Predicting Bicycle Rental:** proyek utuh memprediksi sewa sepeda: korelasi, regresi linear, RMSE, feature engineering.
- **Bab 8 — Python Project for Data Science:** membuat dashboard interaktif dengan Plotly Dash.

ANALOGI SEDERHANA — membaca modul ini seperti naik tangga: setiap bab adalah satu anak tangga. Bab 1–2 menjawab *data science itu apa*, bab 3–5 menyiapkan *perkakasnya*, bab 6 mengasah *otak statistiknya*, bab 7 *praktik proyek beneran*, dan bab 8 *memamerkan hasilnya* dalam bentuk dashboard.

Bab 1 — The Data Science Profession: Profesi Data Science

Data science adalah gabungan ilmu statistika, pemrograman, dan pemahaman bisnis untuk mengubah data mentah menjadi keputusan yang berguna. Sederhananya: data science = cara sistematis mengambil *jawaban* dari tumpukan *angka*.

ANALOGI SEDERHANA — data mentah itu seperti bahan mentah di dapur (beras, cabai, minyak). Data scientist adalah kokinya: ia membersihkan, meracik, memasak, lalu menyajikan menjadi hidangan (wawasan/keputusan) yang bisa dinikmati perusahaan.

Data = kumpulan fakta yang bisa diolah komputer, misalnya angka penjualan atau kategori warna.

Wawasan (insight) = pemahaman baru yang didapat dari data, misalnya 'jualan naik tiap Jumat'.

Supervised learning: belajar dengan kunci jawaban

Supervised learning (pembelajaran terbimbing) adalah cara komputer belajar dari contoh yang sudah berlabel, yaitu contoh yang jawabannya sudah diketahui. Komputer mempelajari pola dari contoh-contoh itu, lalu menebak jawaban untuk data baru yang belum berlabel.

ANALOGI SEDERHANA — seperti murid belajar matematika dengan buku yang ada kunci jawabannya di belakang. Setelah mengerjakan ratusan soal ber-kunci, ia bisa mengerjakan soal ujian yang belum pernah dilihat.

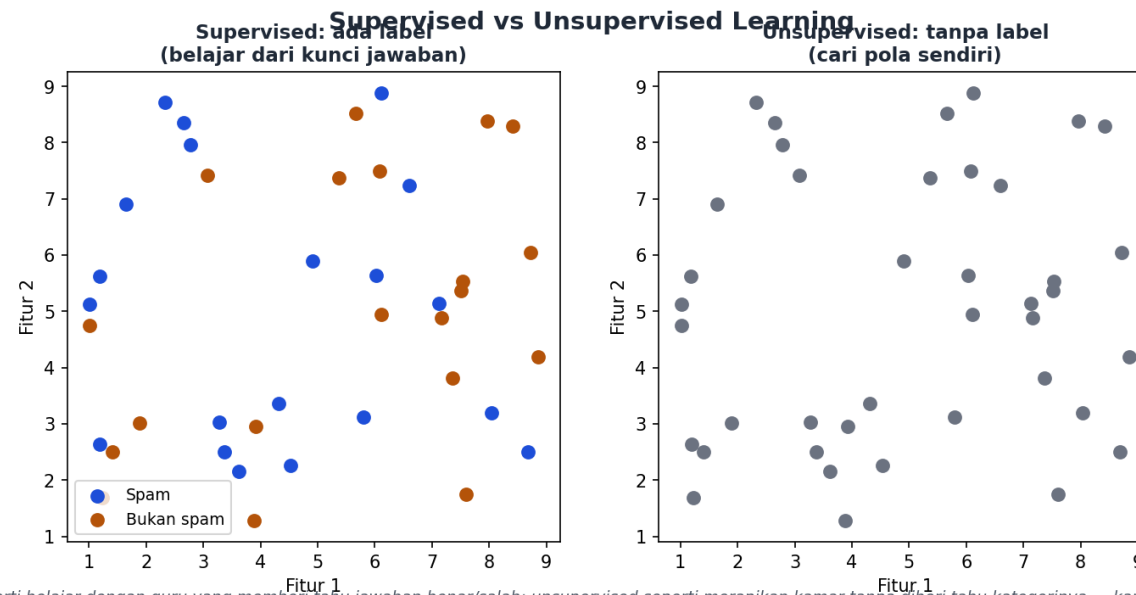
- **Klasifikasi:** menebak *kategori*. Contoh: email ini spam atau bukan? Foto ini kucing atau anjing? Jawabannya selalu berupa label kelompok.
- **Regresi:** menebak *angka*. Contoh: berapa harga rumah ini? Berapa sepeda yang akan disewa besok? Jawabannya berupa bilangan yang kontinu.

Unsupervised learning: belajar tanpa kunci jawaban

Unsupervised learning (pembelajaran tak terbimbing) adalah cara komputer mencari pola sendiri dari data yang *tidak* berlabel. Tidak ada yang memberi tahu jawaban benar; komputer harus menemukan struktur tersembunyi.

ANALOGI SEDERHANA — seperti disuruh merapikan kamar berantakan tanpa diberi tahu kategorinya. Kamu sendiri yang menemukan bahwa kaus kaki cocok dikelompokkan dengan kaus kaki, buku dengan buku.

- **Clustering** (pengelompokan): mengelompokkan data yang mirip. Contoh: mengelompokkan pelanggan toko menjadi 'pembeli hemat', 'pembeli boros', 'pembeli musiman' berdasarkan pola belanja mereka.



Analogi: supervised seperti belajar dengan guru yang memberi tahu jawaban benar/salah; unsupervised seperti merapikan kamar tanpa diberi tahu kategorinya — kamu yang menemukan polanya.

Kiri: supervised (data berlabel warna). Kanan: unsupervised (tanpa label, pola dicari sendiri).

Tahapan proses data science

- **1. Problem definition** — merumuskan pertanyaan bisnis dengan jelas. Contoh: 'berapa sepeda akan disewa besok?'
- **2. Data collection** — mengumpulkan data yang relevan (file CSV, database, API).
- **3. Data cleaning** — membersihkan data: membuang yang rusak, mengisi yang kosong, menyeragamkan format.
- **4. Analysis / modeling** — menganalisis dan melatih model untuk menjawab pertanyaan.
- **5. Communication** — menyampaikan hasil lewat laporan atau presentasi yang mudah dipahami.

ANALOGI SEDERHANA — tahapan ini seperti kerja detektif: pahami kasusnya dulu (1), kumpulkan bukti (2), rapikan bukti yang acak-acakan (3), pecahkan misterinya (4), lalu laporkan ke klien (5).

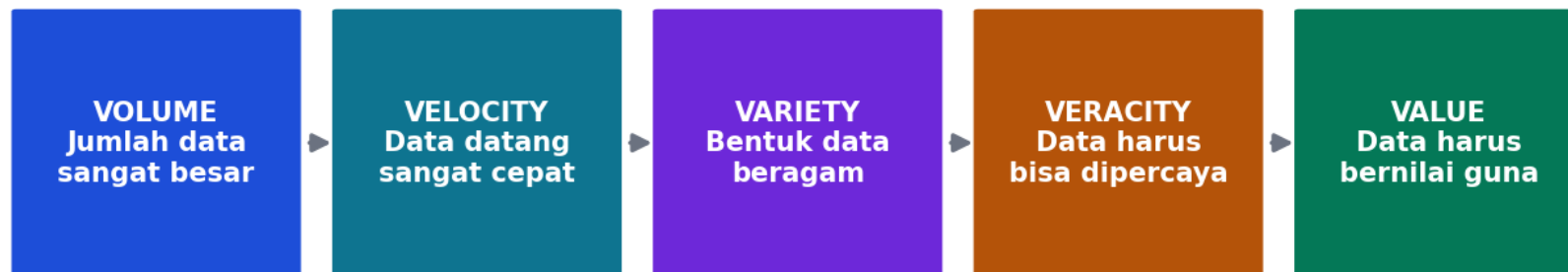
Bab 2 — What is Data Science?: Big Data dan Kawan-kawannya

Istilah **data scientist** populer sekitar tahun 2009–2011 (dipopulerkan DJ Patil dan Andrew Gelman). Sejak itu, minat terhadap data science meledak karena data yang dihasilkan dunia juga meledak — inilah era **big data**.

Big data dan 5V

Big data adalah data yang terlalu besar, terlalu cepat, dan terlalu beragam untuk diolah dengan cara biasa (misalnya Excel). Big data dikenali lewat 5 ciri yang disebut **5V**:

5V Big Data: lima ciri data raksasa



Analogi: big data seperti banjir kiriman paket ke gudang — paketnya banyak (volume), datang terus tanpa henti (velocity), isinya macam-macam (variety), tidak semua asli (veracity), dan hanya berguna kalau isinya dibutuhkan (value).

Lima ciri big data: Volume, Velocity, Variety, Veracity, Value.

- **Volume** (jumlah): datanya sangat banyak. Contoh: jutaan transaksi Gojek setiap hari.
- **Velocity** (kecepatan): data datang sangat cepat dan terus-menerus. Contoh: sensor mobil yang mengirim data tiap detik.
- **Variety** (keragaman): bentuknya macam-macam — angka, teks, foto, video, suara.

- **Veracity** (kebenaran): tidak semua data bisa dipercaya; ada yang rusak atau palsu.
- **Value** (nilai): data hanya berguna kalau bisa menghasilkan keputusan atau uang.

Hadoop = perangkat open-source untuk menyimpan dan mengolah data raksasa dengan cara memecahnya ke banyak komputer biasa yang bekerja sama.

ANALOGI SEDERHANA — Hadoop seperti gudang raksasa yang mempekerjakan ratusan kuli: satu tumpukan beras raksasa dibagi ke banyak karung kecil, tiap kuli mengolah karungnya sendiri, lalu hasilnya digabung. Itulah cara Hadoop menaklukkan data yang tidak muat di satu komputer.

Data mining = teknik menggali pola tersembunyi dari data berukuran besar.

ANALOGI SEDERHANA — data mining seperti menambang emas: dari satu gunung batu (data mentah) yang digali, hanya sedikit bongkahan emas (pola berharga) yang ditemukan — tapi nilainya besar.

Cloud = menyewa komputer raksasa lewat internet, jadi tidak perlu membeli server sendiri.

ANALOGI SEDERHANA — cloud seperti menyewa genset raksasa saat hajatan: kamu pakai listriknya saja, tidak perlu membeli dan merawat gensetnya.

Machine learning vs deep learning

Machine learning adalah cabang AI di mana komputer belajar pola dari data, bukan dari aturan yang ditulis manual. **Deep learning** adalah machine learning yang memakai **jaringan saraf tiruan** berlapis-lapis (neural network) — sangat kuat untuk gambar, suara, dan bahasa.

ANALOGI SEDERHANA — machine learning seperti murid yang belajar dari banyak contoh soal. Deep learning seperti murid yang sama tapi otaknya dibuat berlapis-lapis (seperti kue lapis): lapisan pertama melihat garis, lapisan berikut melihat bentuk, lapisan akhir mengenali wajah. Makin banyak lapisan, makin rumit pola yang bisa dipelajari — tapi butuh data dan komputer yang besar.

- **GAN dan VAE**: model generatif — bisa *membuat* data baru yang mirip aslinya (misalnya membuat foto wajah palsu yang realistis).
- **Contoh nyata**: software Facebook yang memilih iklan untukmu memakai machine learning dari perilakumu.

Aplikasi dan karir

- **Kesehatan**: sistem data science membantu dokter mendapat info penyakit terbaru dan menyusun rencana perawatan personal.
- **Bisnis/retail**: menganalisis kebiasaan belanja pelanggan untuk promosi yang tepat sasaran.

- **Transportasi**: menganalisis data operasional untuk menemukan rute yang macet.
- **Skill wajib**: programming, matematika (aljabar, kalkulus, peluang, statistika), database — plus rasa ingin tahu, eksperimen, dan kemampuan **storytelling** (menceritakan angka menjadi cerita yang dimengerti orang).

ANALOGI SEDERHANA — storytelling dalam data science seperti menjadi penerjemah: angka-angka bicara bahasa mesin, direktur bicara bahasa bisnis. Data scientist menerjemahkan 'R-kuadrat 0,87' menjadi 'kalau suhu naik 5 derajat, sewa sepeda naik sekitar 800 unit'.

Bab 3 — Tools for Data Science: Perkakas Data Scientist

Data scientist tidak bekerja dengan tangan kosong. Ada **bahasa pemrograman** (cara bicara ke komputer), **library** (paket fungsi siap pakai), dan **tools** (lingkungan kerja).

Bahasa-bahasa populer

- **Python**: bahasa paling populer untuk data science. Sintaksnya mirip bahasa Inggris, library-nya paling lengkap. Kalau data science adalah negara, Python adalah bahasa nasionalnya.
- **R**: bahasa favorit ahli statistika, kuat untuk analisis statistik dan grafik (dibantu RStudio).
- **Julia**: bahasa baru yang cepat, dirancang untuk komputasi numerik berat.
- **SQL**: bahasa khusus untuk bertanya ke **database** (gudang data terstruktur). Contoh: `SELECT * FROM penjualan WHERE kota = 'Jakarta'` artinya 'tampilkan semua data penjualan yang kotanya Jakarta'. Sekali bisa SQL, bisa dipakai ke banyak database.

Library Python andalan

Library = kumpulan fungsi siap pakai yang dibuat orang lain, tinggal dipanggil — seperti membeli bumbu instan daripada meracik sendiri.

- **pandas**: mengolah data tabel (baris-kolom) dengan mudah. Analogi: Excel super yang bisa menampung jutaan baris dan dioperasikan dengan kode. Fungsi andalan: `read_csv()` membaca file CSV menjadi tabel.
- **NumPy**: menghitung angka dalam jumlah besar dengan sangat cepat (misalnya rata-rata sejuta angka dalam sekejap). Analogi: kalkulator super untuk array angka.
- **Matplotlib**: menggambar grafik (garis, batang, scatter). Hampir semua grafik Python lahir dari library ini.
- **scikit-learn**: kotak perkakas machine learning siap pakai — regresi, clustering, dan banyak model lain tinggal panggil.

Lingkungan kerja

- **Jupyter Notebook / JupyterLab**: 'buku lab' interaktif — kode, teks penjelasan, dan grafik hidup dalam satu dokumen yang bisa dijalankan per bagian. Berjalan di browser. Dipakai untuk hampir semua final project di spesialisasi ini.
- **RStudio**: lingkungan khusus bahasa R.
- **Visual Studio Code**: editor kode serbaguna yang populer.

Git dan GitHub: mesin waktu untuk kode

Git = sistem yang mencatat setiap perubahan kode, jadi bisa kembali ke versi lama kapan saja.

GitHub = situs untuk menyimpan repository Git secara online sekaligus berkolaborasi.

ANALOGI SEDERHANA — Git seperti mesin waktu untuk skripsimu: setiap **commit** (menyimpan perubahan) adalah titik waktu yang bisa kamu datangi lagi kalau eksperimen barumu merusak segalanya. GitHub adalah lemari arsip online tempat mesin waktu itu disimpan dan bisa dibuka bersama tim. **Pull request** adalah cara mengajukan 'tolong gabungkan perbaikanku ke kode utama', dan **fork** adalah memfotokopi repository orang untuk kamu utak-atik sendiri.

Anatomi final project: notebook DataScienceEcosystem

Final project Course 3 meminta notebook **DataScienceEcosystem.ipynb** berisi 10 cell yang dieksekusi:

- Cell 1–2: judul (heading H1 dengan sintaks #) dan kalimat pengantar (markdown).
- Cell 3: daftar **objectives** (tujuan) dalam bullet markdown.
- Cell 4–5: **ordered list** (daftar bernomor) bahasa populer (Python, R, SQL, Julia) dan library (pandas, NumPy, Matplotlib).
- Cell 6: **tabel** markdown berisi tools (JupyterLab, RStudio, VS Code).
- Cell 7–9: contoh ekspresi aritmetika Python: $(3*4)+5$ menghasilkan 17, dan $200/60$ mengubah 200 menit menjadi jam (3,33).
- Cell 10: nama penulis sebagai heading H2.

Markdown = cara menulis teks berformat (tebal, judul, tabel) dengan simbol sederhana, dipakai di cell teks notebook.

Cell = satu kotak di notebook — bisa berisi teks (markdown) atau kode Python yang bisa dijalankan.

Bab 4 — Problems, Algorithms and Flowcharts: Berpikir seperti Komputer

Apa itu problem dan algoritma?

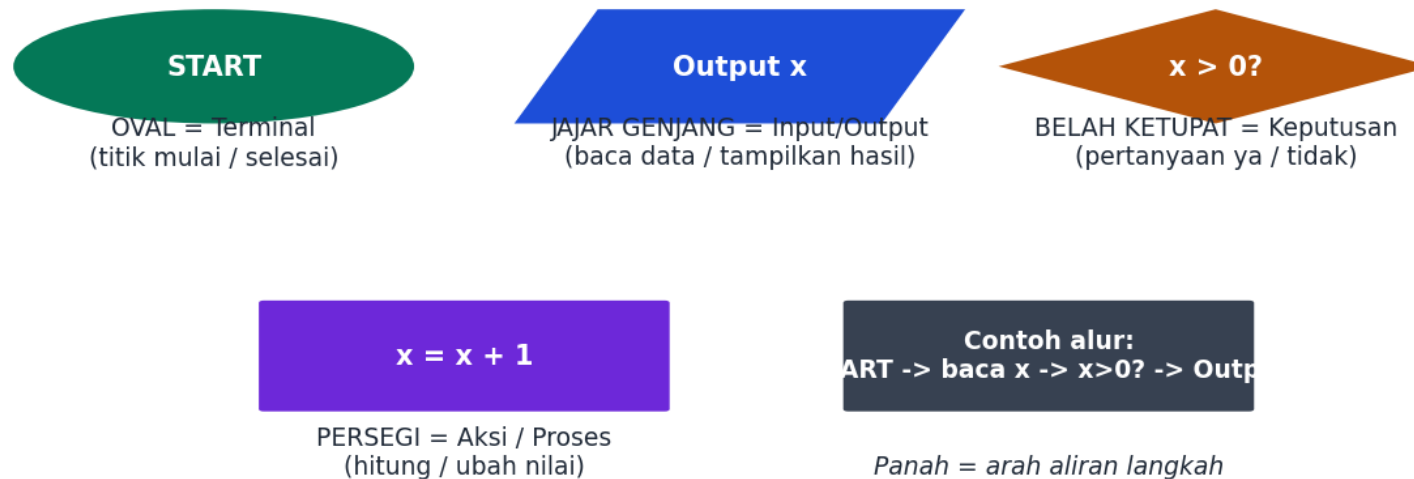
Dalam ilmu komputer, **problem** adalah pertanyaan tentang data input yang bisa dinyatakan secara matematis beserta tipe output yang diharapkan. Contoh: 'diberi list berisi n bilangan bulat, berapa indeks dari bilangan x ?' — output-nya bertipe **integer** (bilangan bulat). Pertanyaan 'apakah x ada di list?' output-nya bertipe **boolean** (benar/salah).

ANALOGI SEDERHANA — problem seperti pesanan di restoran: 'saya mau nasi goreng' (input: keinginan) harus jelas hasilnya apa (output: sepiring nasi goreng), bukan sekadar 'saya lapar'. Komputer hanya bisa bekerja kalau pesannya jelas.

Algoritma adalah sekumpulan instruksi langkah demi langkah yang umum, sederhana, dan **pasti berhenti** dalam jumlah langkah berhingga. Algoritma bukan sekadar program komputer — ia adalah *idenya*, sedangkan program adalah idenya yang ditulis dalam bahasa pemrograman tertentu.

ANALOGI SEDERHANA — algoritma adalah resep masakan: daftar langkah yang jelas ('tumis bawang 2 menit') dan pasti selesai (tidak ada langkah 'tumis selamanya'). Resepnya sama walau ditulis dalam bahasa Indonesia atau Inggris — begitu juga algoritma, sama walau diimplementasikan dalam Python atau Java.

Empat Simbol Flowchart yang Wajib Hafal



Empat simbol flowchart: oval (terminal), jajar genjang (I/O), belah ketupat (keputusan), persegi (proses).

Membaca flowchart: contoh telusur

Perhatikan flowchart: START → 'Get x' → belah ketupat 'Is x > 0?' → jika YA: jajar genjang 'Output TRUE' → FINISH; jika TIDAK: 'Output FALSE' → FINISH. Jika x = 2, maka 2 > 0 itu benar, jadi output-nya **True**. Kunci membaca flowchart: ikuti panahnya satu per satu seperti mengikuti peta, jangan melompat.

Pseudocode: resep dalam bahasa sehari-hari

Pseudocode adalah cara menulis algoritma dengan bahasa yang mirip bahasa manusia (campur sedikit gaya kode), tanpa aturan sintaks ketat. Dipakai karena lebih **universal** — dimengerti siapa saja tanpa harus bisa bahasa pemrograman tertentu.

ANALOGI SEDERHANA — pseudocode seperti menulis resep dengan bahasamu sendiri sebelum memasak serius: 'ambil 3 telur, kocok sampai rata, goreng 3 menit'. Tidak perlu format buku resep resmi — yang penting langkahnya jelas.

```
x ← 3
for 0 ≤ i < 3 do
  x ← x + i
end for
# telusur: i=0 -> x=3 | i=1 -> x=4 | i=2 -> x=6
# hasil akhir x = 6
```

Contoh telusur lain: $y \leftarrow 1$; $i \leftarrow 0$; while $i < 4$ do $y \leftarrow 2 \times y$; $i \leftarrow i+1$ — y berlipat ganda 4 kali: $1 \rightarrow 2 \rightarrow 4 \rightarrow 8 \rightarrow 16$, jadi hasil akhir **$y = 16$** . Dan loop while $x < 0$ dengan $x = 1$ tidak pernah jalan ($1 < 0$ itu salah), jadi **tidak** infinite.

Dua algoritma klasik

- **Algoritma Euclid**: mencari **FPB** (faktor persekutuan terbesar) dua bilangan dengan cara membagi berulang memakai sisa. Contoh: $\text{FPB}(48, 18) = 6$. Analogi: memotong dua tali (48 cm dan 18 cm) menjadi potongan-potongan sama panjang yang paling panjang — jawabannya 6 cm.
- **Metode Heron**: cara kuno menaksir akar kuadrat dengan tebakan yang diperbaiki berulang-ulang. Untuk $x = 5$: mulai $g = 5 \rightarrow$ rata-ratakan 5 dan $5/5=1$ jadi 3 $\rightarrow$ rata-ratakan 3 dan $5/3$ jadi $7/3 \rightarrow 5/(7/3)$ dan $7/3$ sudah sama jika dibulatkan ke 0 desimal $\rightarrow$ berhenti. Hasil: **$7/3$** ($\approx 2,33$, dekat dengan $\sqrt{5}$). Analogi: seperti menebak berat semangka — tebak, timbang selisihnya, perbaiki tebakan, ulangi sampai pas.

Bab 5 — Python for Data Science: Dasar Python

Python adalah bahasa utama di spesialisasi ini. Berikut fondasi yang dipakai di semua course.

Variabel: kotak berlabel

```
x = 10          # simpan angka 10 ke kotak bernama x
nama = "Baghiz" # simpan teks ke kotak bernama nama
# Aturan nama: boleh huruf/angka/underscore (my_age), TIDAK boleh spasi (my age)
# atau simbol (my+age). Contoh valid: x, my_age
```

ANALOGI SEDERHANA — variabel seperti kotak berlabel di gudang: labelnya 'x', isinya 10. Kamu bisa mengganti isinya kapan saja tanpa mengganti labelnya.

List dan indexing: deretan kotak bernomor

```
ages = [22, 15, 53, 6, 36]
print(ages[1 + 3]) # ages[4] = 36
print(ages[-1])   # elemen terakhir = 36
print(ages[3])    # = 6
print(sum(ages))  # jumlahkan semua = 112
```

ANALOGI SEDERHANA — list seperti deretan loker bernomor di stasiun — bedanya, nomor loker Python mulai dari **0**, bukan 1. `ages[0]` adalah loker pertama (22). Indeks negatif menghitung dari belakang: `ages[-1]` = loker terakhir.

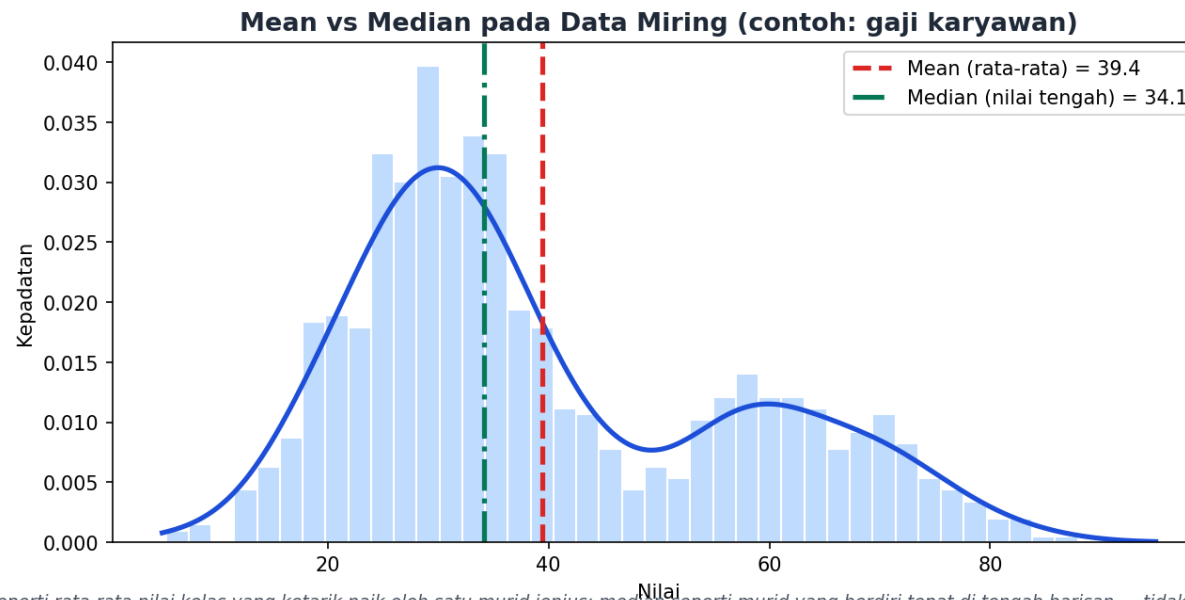
- Membuat list: `ages = [34, 55, 89, 10]` (kurung siku). Kurung biasa (`34, 55, 89, 10`) itu tuple, bukan list.
- `print()` menampilkan ke layar; `sum()` menjumlahkan isi list.
- **Perulangan for:** `for umur in ages: print(umur)` — kerjakan sesuatu untuk tiap isi list, seperti memeriksa tiap loker satu per satu.

Bab 6 — Statistics and Clustering in Python: Statistik & Pengelompokan

Statistika adalah ilmu meringkas data menjadi angka-angka yang bermakna. Clustering adalah teknik mengelompokkan data yang mirip. Keduanya memakai Python (NumPy, Matplotlib, scikit-learn).

Mean, median, modus: tiga cara mencari 'nilai tengah'

- **Mean** (rata-rata): jumlah semua nilai dibagi banyaknya data. Contoh: umur 18, 22, 23, 50, 34, 29 → jumlah 176 dibagi 6 = **29,3**.
- **Median**: nilai tengah setelah data diurutkan. Tahan terhadap nilai ekstrem.
- **Modus**: nilai yang paling sering muncul.



Analogi: mean seperti rata-rata nilai kelas yang ditarik naik oleh satu murid jenius; median seperti murid yang berdiri tepat di tengah barisan — tidak terpengaruh nilai ekstrem.

Pada data miring, mean ditarik oleh nilai ekstrem sedangkan median tetap di tengah.

ANALOGI SEDERHANA — mean seperti rata-rata nilai kelas yang ikut naik karena satu murid jenius dapat 100. Median seperti murid yang berdiri tepat di tengah barisan dari terpendek ke tertinggi — satu murid jenius tidak menggeser posisinya.

Varians dan standar deviasi: seberapa menyebar data

Varians mengukur seberapa jauh tiap data menyebar dari rata-rata. **Standar deviasi (SD)** adalah akar dari varians — satuannya sama dengan data aslinya sehingga lebih mudah dibaca. Jaraknya **dikuadratkan** dulu karena yang penting adalah *besarnya* selisih, bukan arahnya (di atas atau di bawah rata-rata sama-sama dihitung positif).

ANALOGI SEDERHANA — seperti lomba menembak target: mean adalah titik tengah target, tiap tembakan adalah data. Varians/SD mengukur seberapa menyebar lubang tembakan — penembak jitu punya SD kecil (rapat di tengah), penembak amatir punya SD besar (menyebar ke mana-mana).

```
# Contoh: berat 65, 90, 50, 75 kg
mean = (65+90+50+75)/4 = 70
selisih kuadrat = (65-70)^2+(90-70)^2+(50-70)^2+(75-70)^2 = 850
varians = 850/(4-1) = 283,3 (dibagi n-1, bukan n)
standar deviasi = akar(283,3) = 16,83
```

Populasi vs sampel

Populasi adalah seluruh anggota kelompok yang diteliti; **sampel** adalah sebagian yang dipakai untuk mewakili populasi. Sampel dipakai karena lebih **praktis** — meneliti semua orang mahal dan lama. Contoh: populasi = semua murid baru Victoria High School, sampel = 150 murid yang dipilih acak untuk disurvei.

ANALOGI SEDERHANA — seperti mencicip sup: kamu tidak perlu menghabiskan sepanci untuk tahu rasanya — sesendok (sampel) yang diaduk rata cukup mewakili sepanci (populasi).

NumPy: kalkulator array super cepat

```
import numpy as np
data = np.array([[1,2],[3,4],[5,6]])
data[:, 1]      # kolom ke-1 -> [2, 4, 6]
data[:, -1]     # kolom terakhir -> [2, 4, 6]
np.mean(data)   # rata-rata SEMUA nilai (bukan per kolom!)
np.mean(data, 0) # rata-rata per kolom (axis=0)
np.min(data, 0)  # nilai minimum per kolom
```

- NumPy itu **high-level** (mudah dipakai), **performant** (sangat cepat), dan **well-tested** (teruji).
- Fungsi statistik bawaan: mean, average, std (standar deviasi), median.
- **Normalisasi**: mengubah data ke skala 0–1 dengan rumus $(x - \min) / (\max - \min)$. Analogi: menyamakan satuan cm dan km sebelum dibandingkan — kalau tidak, yang satuannya besar akan mendominasi.

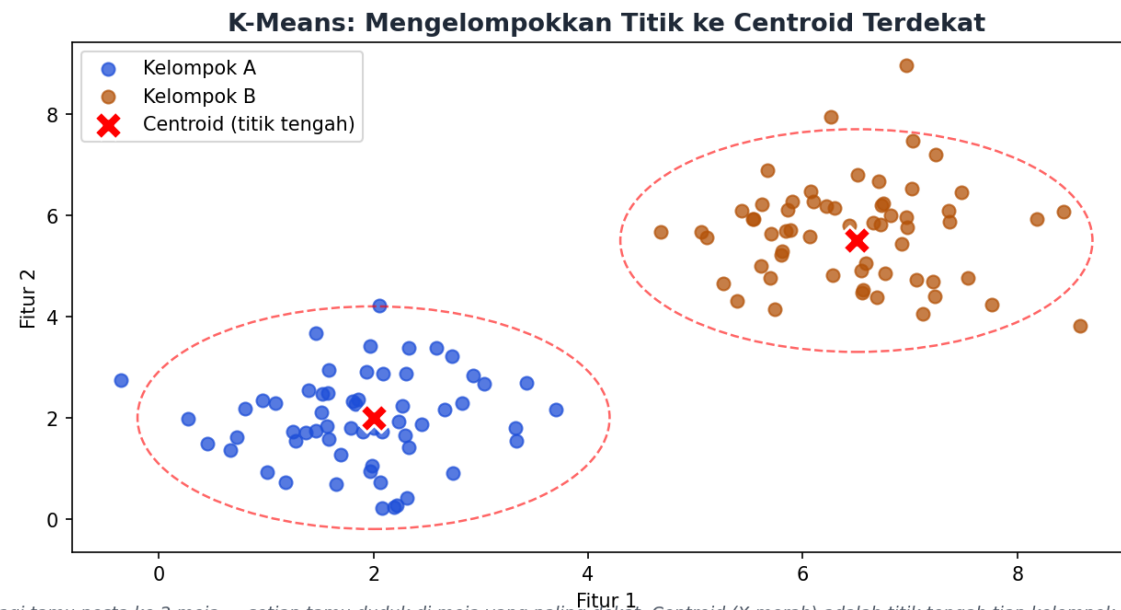
K-means clustering

K-means mengelompokkan data menjadi **k** kelompok (k = jumlah kelompok yang kita tentukan). Caranya: tiap titik diberikan ke kelompok yang **centroid** (titik tengah kelompok) terdekat, lalu centroid dihitung ulang, diulang sampai stabil.

ANALOGI SEDERHANA — seperti membagi tamu pesta ke 3 meja bundar: tiap tamu duduk di meja terdekat, lalu tiap meja menghitung titik tengah anggotanya dan tamu boleh pindah ke meja yang kini lebih dekat. Diulang sampai tidak ada yang pindah meja.

Jarak Euclidean = jarak garis lurus antara dua titik, seperti diukur dengan penggaris. Rumus: akar dari jumlah kuadrat selisih tiap koordinat.

```
from sklearn.cluster import KMeans
from numpy.linalg import norm
jarak = norm(x1 - x2) # jarak Euclidean, aman untuk nilai negatif
km = KMeans(n_clusters=2, n_init=10, random_state=42)
km.fit(data_standar)
```



Analogi: seperti membagi tamu pesta ke 2 meja — setiap tamu duduk di meja yang paling dekat. Centroid (X merah) adalah titik tengah tiap kelompok, dihitung ulang sampai stabil.

K-means dengan $k=2$: tiap titik ikut centroid (X merah) terdekat.

Scatter plot dan studi kasus banknote

Scatter plot menggambar tiap data sebagai titik di koordinat (x, y) — cara tercepat melihat pola. Di Matplotlib: `plt.scatter(x, y)`; parameter `alpha` mengatur transparansi titik, `s` mengatur ukuran titik.

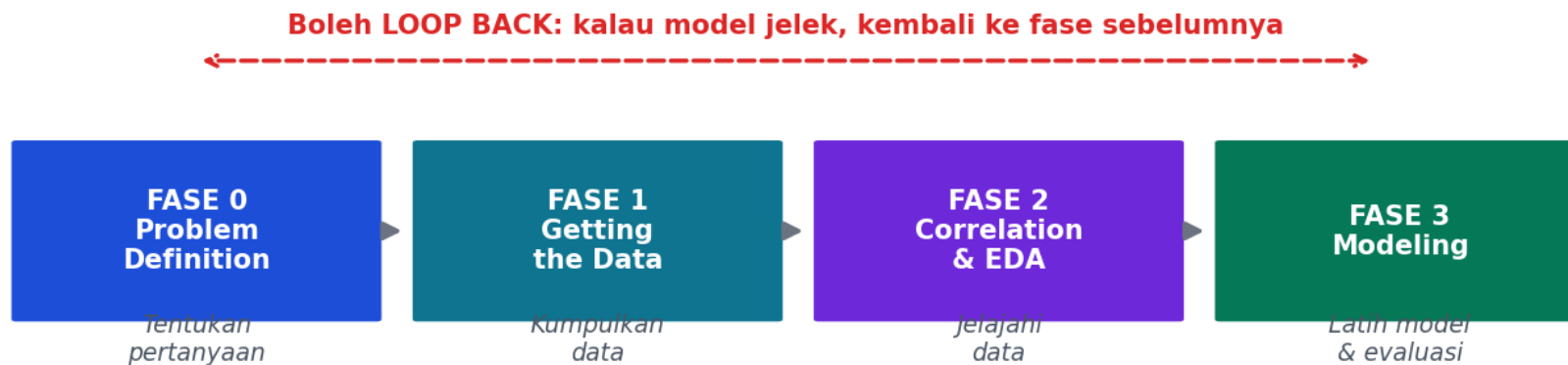
Studi kasus nyata (project Course 6): dataset **autentikasi uang kertas** — 1372 lembar uang, tiap lembar diukur 4 fitur citra (variance, skewness, curtosis, entropy), label asli (762) vs palsu (610). Hasil: K-means k=2 hanya cocok 55,9% dengan label asli — nyaris seperti tebak-tebakan!

ANALOGI SEDERHANA — pelajarannya seperti memisahkan kelereng asli dan palsu hanya dengan melihat warnanya: kalau warnanya tumpang tindih, pengelompokan buta akan gagal. Tidak semua data cocok untuk K-means — itulah gunanya EDA (exploratory data analysis): kenali datamu dulu sebelum memilih alat.

Bab 7 — Capstone: Predicting Bicycle Rental (Proyek Akhir)

Proyek capstone memakai dataset sewa sepeda harian (**day.csv**) untuk menjawab satu pertanyaan bisnis: **berapa sepeda yang akan disewa besok?** Kolom penting: cnt (jumlah disewa = target), atemp (suhu terasa), weathersit (kondisi cuaca 1–4), dteday (tanggal). Tools: Python (pandas, Matplotlib, NumPy).

Empat Fase Proyek Data Science (Capstone Sepeda)



Analogi: seperti membangun rumah — gambar denah dulu (fase 0), kumpulkan bahan (fase 1), cek kualitas bahan (fase 2), baru bangun (fase 3). Kalau atap bocor, kembali perbaiki.

Empat fase proyek capstone — boleh loop back kalau model belum bagus.

Fase 1 — Getting the data (mengambil data)

```
import pandas as pd
df = pd.read_csv("day.csv")          # baca file menjadi tabel
df["date"] = pd.to_datetime(df["dteday"]) # ubah teks tanggal jadi tipe tanggal
```

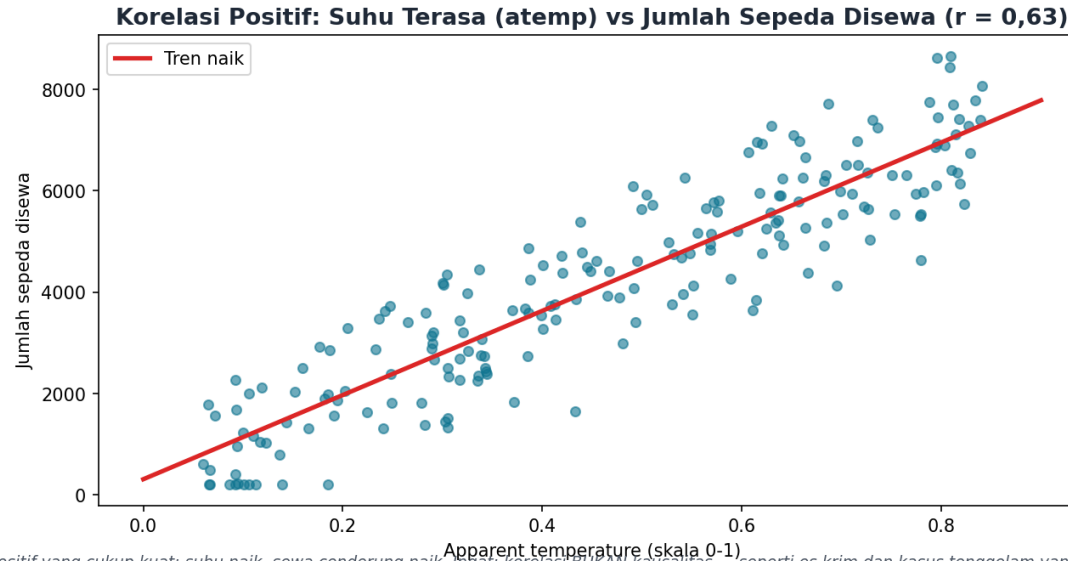
Pandas dipanggil dengan nama panggilan `pd` — itu singkatan standar di industri, bukan sekadar tradisi.

Fase 2 — Korelasi: mencari hubungan antar kolom

Korelasi mengukur seberapa kuat dua hal bergerak bersama. **Koefisien korelasi (r)** nilainya dari -1 sampai 1: **1** = positif sempurna (naik bareng dengan rasio sama), **-1** = negatif sempurna (satu naik, satu turun), **0** = tidak berhubungan. Contoh: $r = -0,7$ berarti korelasi negatif yang kuat.

ANALOGI SEDERHANA — korelasi seperti dua orang dansa: $r = 1$ mereka melangkah seirama ke arah yang sama; $r = -1$ mereka melangkah berlawanan arah; $r = 0$ mereka dansa sendiri-sendiri tanpa peduli pasangan.

Temuan EDA: rata-rata sewa saat cuaca 1 (cerah) = **4876**, cuaca 2 (berawan) = **4035**, cuaca 3 jauh lebih rendah. Korelasi `cnt` vs `atemp` = **0,63** (positif, cukup kuat), dan suhu terasa (apparent temperature) ternyata **lebih prediktif** daripada suhu biasa.



$r = 0,63$ artinya korelasi positif yang cukup kuat: suhu naik, sewa cenderung naik. Ingat: korelasi BUKAN kausalitas — seperti es krim dan kasus tenggelam yang sama-sama naik saat cuaca panas.

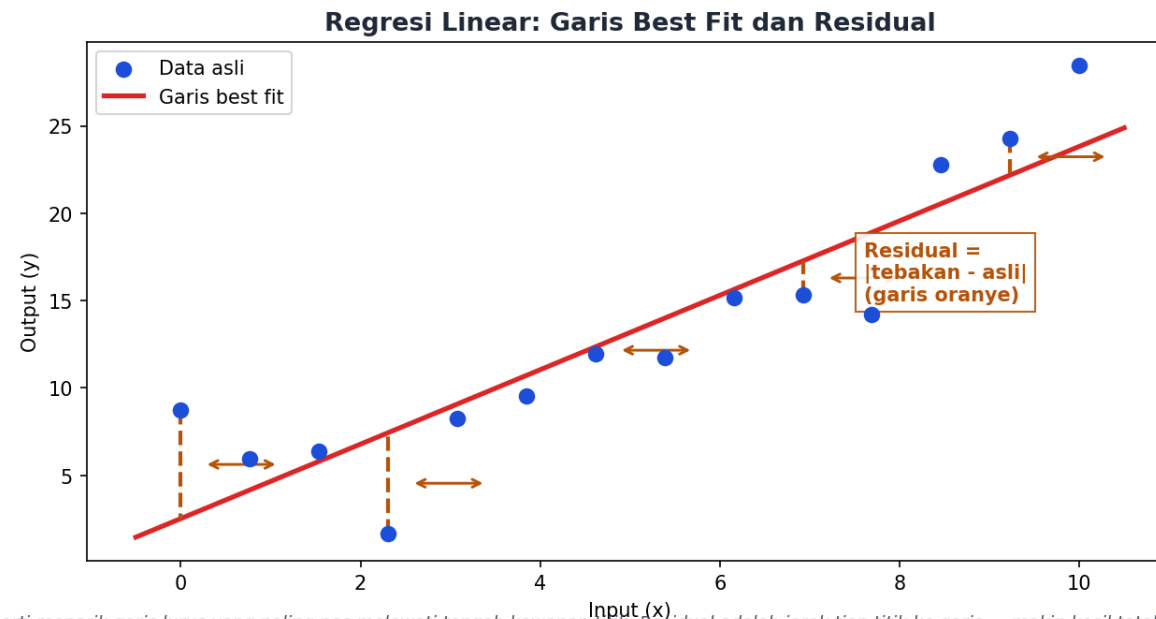
Makin hangat suhu terasa, makin banyak sepeda disewa ($r = 0,63$).

Korelasi bukan kausalitas = dua hal yang bergerak bersama belum tentu saling menyebabkan.

ANALOGI SEDERHANA — penjualan es krim dan kasus tenggelam sama-sama naik saat musim panas (korelasi kuat), tapi es krim tidak *menyebabkan* orang tenggelam — penyebab sebenarnya adalah cuaca panas. Jangan sampai modelmu menyimpulkan 'larang es krim agar orang selamat'.

Fase 3 — Modeling: regresi linear

Regresi linear menebak angka target dengan menarik **garis best fit** (garis lurus yang paling pas melewati tengah data). Asumsinya: hubungan input-output itu linear plus sedikit noise (gangguan acak). Garis terbaik ditemukan dengan memperkecil **residual** — selisih tiap tebakan dengan data asli (dikuadratkan agar positif). Kalau inputnya banyak, garisnya menjadi **hyperplane** di dimensi lebih tinggi; tekniknya disebut **regresi multilinear**.



Analogi: best fit seperti menarik garis lurus yang paling pas melewati tengah kawanan titik. Residual adalah jarak tiap titik ke garis — makin kecil totalnya, makin bagus garisnya.

Garis best fit meminimalkan total residual (garis oranye putus-putus).

```
from sklearn.linear_model import LinearRegression
lr = LinearRegression()
# reshape(-1,1): ubah jadi format kolom agar cocok untuk fit()
lr.fit(df['atemp'].values.reshape(-1,1), df['cnt'].values.reshape(-1,1))
# lr.fit() = latih model dari data input dan output
```

Train/validation split dan RMSE: menguji dengan jujur

Data dibagi dua: **training set** untuk melatih model, **validation set** (data yang belum pernah dilihat model) untuk menguji. Di proyek ini batasnya tanggal **2012-06-01**: data sebelumnya untuk latihan, sesudahnya untuk ujian.

ANALOGI SEDERHANA — seperti murid: latihan soal di rumah (training), lalu ujian dengan soal baru (validation). Nilai ujian yang jujur hanya didapat dari soal yang belum pernah dilihat — kalau ujiannya pakai soal latihan yang sama, nilainya menipu.

RMSE (Root Mean Squared Error) = rata-rata kesalahan tebakan model (makin kecil makin bagus). Dihitung: rata-ratakan kuadrat selisih tebakan vs asli, lalu akarkan.

```
import numpy as np
rmse = np.sqrt((lr.predict(validation_inputs) - validation_outputs)**2).mean())
# RMSE dihitung di VALIDATION set, bukan training set
```

Hasil model awal: RMSE sekitar **2000** dari skala sewa ~8000 per hari — buruk. Model bagus di data latihan tapi jelek di data ujian (**overfit**: menghafal, bukan memahami). Masalah utamanya: model tidak memakai cukup banyak fitur input.

Feature engineering: fitur last_week

Feature engineering = membuat kolom/fitur baru dari data lama agar model lebih pintar.

Solusinya: tambah fitur **last_week** = rata-rata sewa 7 hari terakhir untuk tiap hari, karena jumlah sewa berubah mengikuti tren waktu:

```
df['last_week'] = (df['cnt'].cumsum() - df['cnt'].cumsum().shift(7)) / 7
df = df.dropna() # buang baris yang kosong (NaN) akibat shift 7 hari
```

ANALOGI SEDERHANA — seperti meramal omzet warung besok: selain melihat cuaca, kamu juga melihat rata-rata omzet seminggu terakhir — trennya memberi petunjuk yang tidak dimiliki data cuaca saja.

Hasil akhir proyek: model multilinear dengan fitur tambahan memprediksi jauh lebih baik. Pelajaran capstone: (1) pahami masalah dulu (fase 0), (2) EDA sebelum modeling, (3) korelasi bukan kausalitas, (4) uji model di data baru (validation), (5) kalau model jelek, loop back — tambah fitur, bukan menyerah.

Bab 8 — Python Project for Data Science: Dashboard Interaktif

Dashboard adalah halaman berisi kumpulan grafik dan angka penting yang bisa diklik dan dijelajah — seperti panel speedometer mobil: sekilas langsung tahu kecepatan, bensin, dan suhu mesin perusahaanmu.

Plotly Dash = library Python untuk membuat dashboard web interaktif tanpa harus bisa HTML/JavaScript.

Aplikasi Dash punya dua bagian: **layout** (tata letak: grafik apa saja yang tampil) dan **callback** (aturan interaktivitas: 'kalau pengguna memilih X, grafik Y berubah').

ANALOGI SEDERHANA — callback seperti sakelar lampu: kamu tidak menyalakan lampu secara manual tiap kali — kamu memasang aturan 'kalau sakelar ditekan, lampu menyala'. Di dashboard: 'kalau pengguna pilih tahun 2020, grafik otomatis menampilkan data 2020'.

```
from dash import Dash, dcc, html
from dash.dependencies import Input, Output
app = Dash(__name__)
app.layout = html.Div([
    dcc.Dropdown(id='pilih-tahun', options=[2020, 2021]),
    dcc.Graph(id='grafik-sewa')
])
@app.callback(Output('grafik-sewa', 'figure'),
              Input('pilih-tahun', 'value'))
def update(than):
    return buat_grafik(than) # grafik dihitung ulang tiap pilihan berubah
```

- **Alur kerja dashboard:** siapkan data (pandas) → buat grafik (Plotly) → susun layout → tambah callback → jalankan sebagai aplikasi web.
- **Contoh proyek:** dashboard penjualan mobil saat resesi — pengguna memilih tahun dan jenis kendaraan, grafik penjualan langsung menyesuaikan. Inilah 'panggung' tempat semua skill bab 1–7 dipamerkan.

Glosarium: Kamus Istilah Cepat

Semua istilah asing yang muncul di modul ini, dikumpulkan di satu tempat.

Akurasi / error = seberapa tepat tebakan model; error adalah besarnya kesalahan.

API = jembatan agar dua software bisa saling bicara dan bertukar data.

Array = deretan angka dalam wadah terstruktur (misalnya tabel angka NumPy).

Boolean = tipe data benar/salah (True/False).

Callback = aturan 'kalau X terjadi, lakukan Y' di aplikasi interaktif.

Centroid = titik tengah sebuah kelompok (cluster).

Clustering = mengelompokkan data yang mirip tanpa diberi label dulu.

Commit = menyimpan satu titik versi perubahan kode di Git.

CSV = file teks berisi tabel dengan koma sebagai pemisah kolom.

Dashboard = halaman berisi grafik interaktif untuk memantau data.

Data mining = menggali pola tersembunyi dari data berukuran besar.

Dataframe = tabel data di pandas (baris + kolom berlabel).

Deep learning = machine learning dengan jaringan saraf berlapis-lapis.

EDA = exploratory data analysis: menjelajahi data sebelum dimodelkan.

Euclidean distance = jarak garis lurus antara dua titik.

Feature = kolom/ciri data yang dipakai model untuk menebak (misalnya suhu).

Feature engineering = membuat fitur baru dari data lama agar model lebih pintar.

Flowchart = diagram alir langkah-langkah algoritma.

Fork = menyalin repository GitHub orang untuk diutak-atik sendiri.

Hadoop = perkakas mengolah data raksasa di banyak komputer sekaligus.

Hyperplane = versi 'garis lurus' di dimensi lebih dari 2.

Integer = bilangan bulat (... , -1, 0, 1, 2, ...).

Klasifikasi = menebak kategori/label (misalnya spam atau bukan).

Korelasi = ukuran seberapa kuat dua hal bergerak bersama (-1 sampai 1).

Label = jawaban/kategori yang sudah diketahui pada data latih.

Library = paket fungsi siap pakai (misalnya pandas, NumPy).

Machine learning = komputer yang belajar pola dari data, bukan dari aturan manual.

Markdown = cara menulis teks berformat dengan simbol sederhana.

Mean / median / modus = rata-rata / nilai tengah / nilai tersering.

Model = rumus tebakan yang dipelajari komputer dari data.

Normalisasi = menyamakan skala data ke rentang 0–1.

Overfit = model yang menghafal data latih tapi gagal di data baru.

Populasi / sampel = seluruh kelompok / sebagian yang mewakili.

Pseudocode = menulis algoritma dengan bahasa sehari-hari.

Pull request = usulan menggabungkan perubahan kode ke repository utama.

Regresi = menebak angka kontinu (misalnya harga, jumlah sewa).

Repository = tempat penyimpanan proyek kode (di Git/GitHub).

Residual = selisih antara tebakan model dan data asli.

RMSE = rata-rata kesalahan tebakan model; makin kecil makin bagus.

Scatter plot = grafik titik-titik untuk melihat hubungan dua variabel.

Standar deviasi = ukuran sebaran data dari rata-rata.

Supervised learning = belajar dari contoh yang sudah berlabel/kunci jawaban.

Training / validation = data untuk melatih model / data baru untuk menguji model.

Unsupervised learning = mencari pola sendiri dari data tanpa label.

Varians = rata-rata kuadrat selisih data dari mean.

Visualisasi = menyajikan data dalam bentuk grafik/gambar.

Penutup

Selamat! Kamu sudah menapaki 8 anak tangga: dari memahami apa itu data science, mengenal big data, menguasai perkakas, berpikir algoritmik, ngoding Python, berhitung statistik, menyelesaikan proyek capstone prediksi sewa sepeda, sampai memamerkannya dalam dashboard.

Tiga bekal terpenting yang dibawa pulang: **(1)** pola pikir fase proyek — definisikan masalah dulu, jangan langsung ngoding; **(2)** korelasi bukan kausalitas dan model wajib diuji di data baru; **(3)** tools hanyalah alat — pandas, scikit-learn, dan Dash semuanya bisa dipelajari, yang mahal adalah kemampuan bertanya dengan benar dan bercerita dengan jujur.

Modul ini disusun dari materi 8 course spesialisasi Data Science Foundations (University of London) di Coursera yang diselesaikan pada Oktober 2026, lengkap dengan kunci jawaban dan koreksi dari hasil pengerjaan nyata tiap quiz.